

Teste Estrutural

Do código ao Grafo de Fluxo de Controle
McCabe e teste de caminho básico

Aula introdutória com exemplos, soluções e exercícios

Objetivos da aula

- Observar um código e identificar fluxo de controle
- Transformar código em Grafo de Fluxo de Controle
- Identificar nós, arestas e decisões
- Calcular a Complexidade Ciclomática de McCabe
- Identificar caminhos linearmente independentes
- Derivar casos de teste para percorrer caminhos

De onde estamos vindo?

No teste funcional, pensamos de fora para dentro.

Entrada -> Software -> Saida

A pergunta principal era: para determinada entrada, qual deveria ser a saida?

Agora mudamos a pergunta

Em vez de perguntar somente se a saída está correta, passamos a perguntar:

O que aconteceu dentro do programa para produzir essa saída?

Teste estrutural

No teste estrutural conhecemos a implementacao.

Observamos:

- instrucoes
- decisoes
- desvios
- repeticoes
- caminhos de execucao

O amadurecimento esperado

O aluno precisa parar de olhar o código como uma lista de linhas.

Ele deve enxergar o código como uma rede de caminhos possíveis.

Linha de código -> bloco -> decisão -> caminho -> caso de teste

Codigo nao e apenas sequencia

Quando programamos, lemos linha por linha.

Quando testamos estruturalmente, perguntamos:

- existe desvio?
- existe retorno antecipado?
- existe loop?
- existe caminho que nunca executa?
- qual entrada percorre cada caminho?

Grafo de Fluxo de Controle

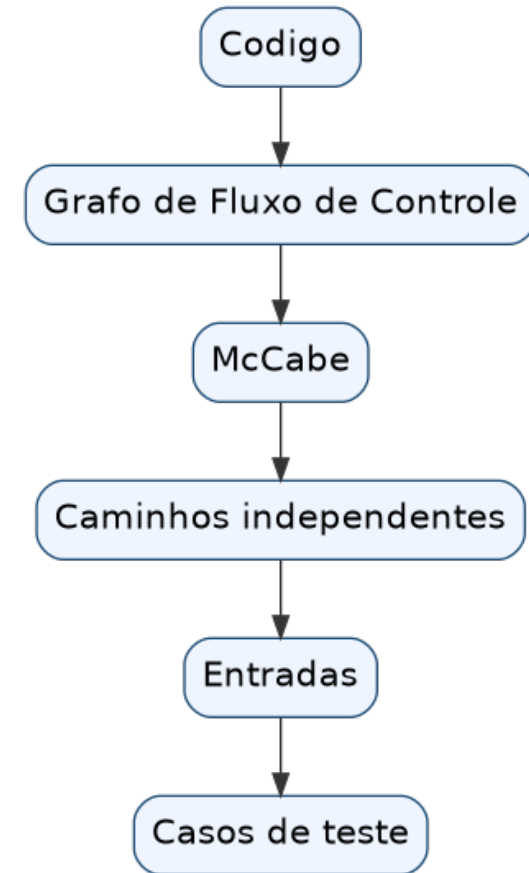
O comportamento do código pode ser representado por um Grafo de Fluxo de Controle.

Nos representam instruções ou blocos.

Arestas representam transferências possíveis de controle.

Processo central da aula

Sempre faremos a transformacao abaixo.

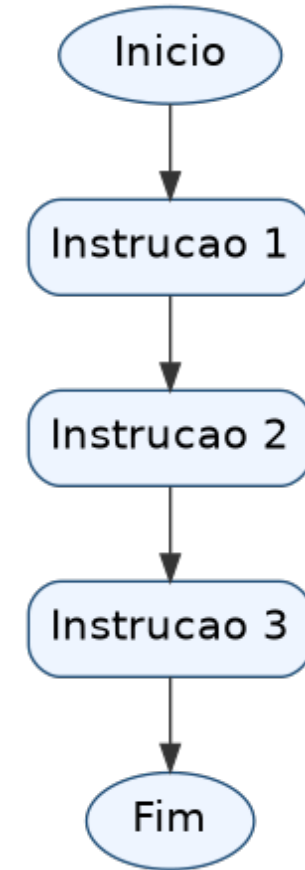


Representacao 1 - Sequencia

Uma sequencia simples nao cria ramificacao.

Tudo que entra no primeiro bloco obrigatoriamente segue para o proximo.

```
int calcular(int x) {  
    int r = x * 2;  
    r++;  
    return r;  
}
```



Leitura estrutural da sequencia

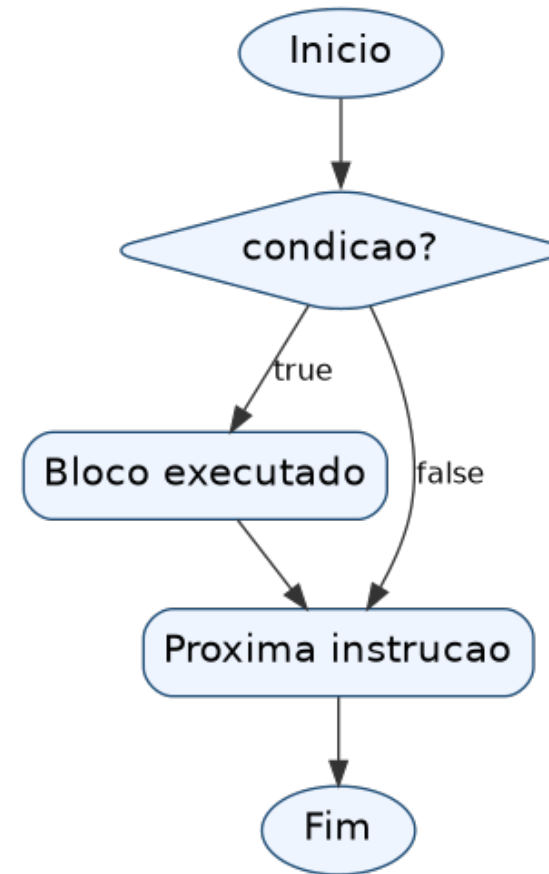
- Não há decisão.
- Existe apenas um caminho principal.
- Um teste já percorre todas as instruções sequenciais.

McCabe: $V(G) = 1$

Representacao 2 - Condicional simples

Um if sem else cria dois caminhos: executar o bloco ou pular o bloco.

```
if (idade >= 18) {  
    autorizado = true;  
}  
return autorizado;
```



Leitura estrutural do if

A condicao gera duas arestas:

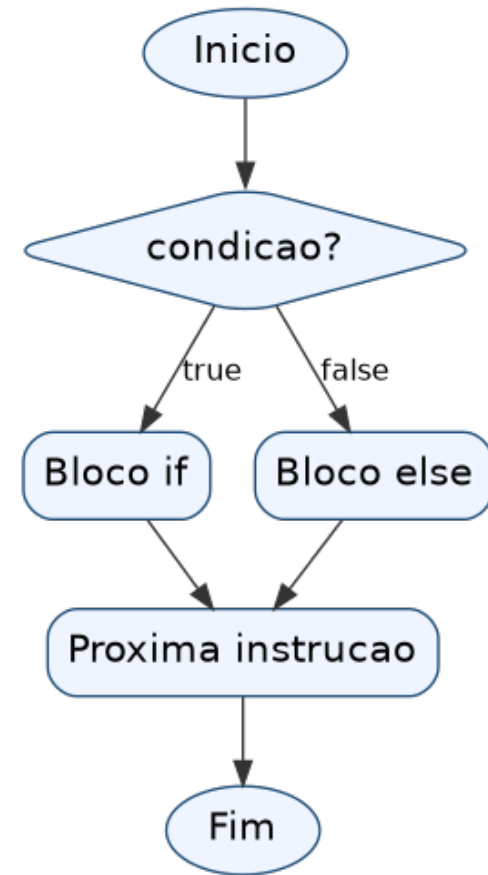
- true: entra no bloco
- false: pula o bloco

McCabe: 1 decisao + 1 = 2

Representacao 3 - if / else

O if/else tambem possui duas saidas, mas ambas possuem bloco explicito.

```
if (a > b) {  
    maior = a;  
} else {  
    maior = b;  
}  
return maior;
```



Leitura estrutural do if/else

Do ponto de vista de fluxo:

- um teste deve fazer a condicao verdadeira;
- outro teste deve fazer a condicao falsa.

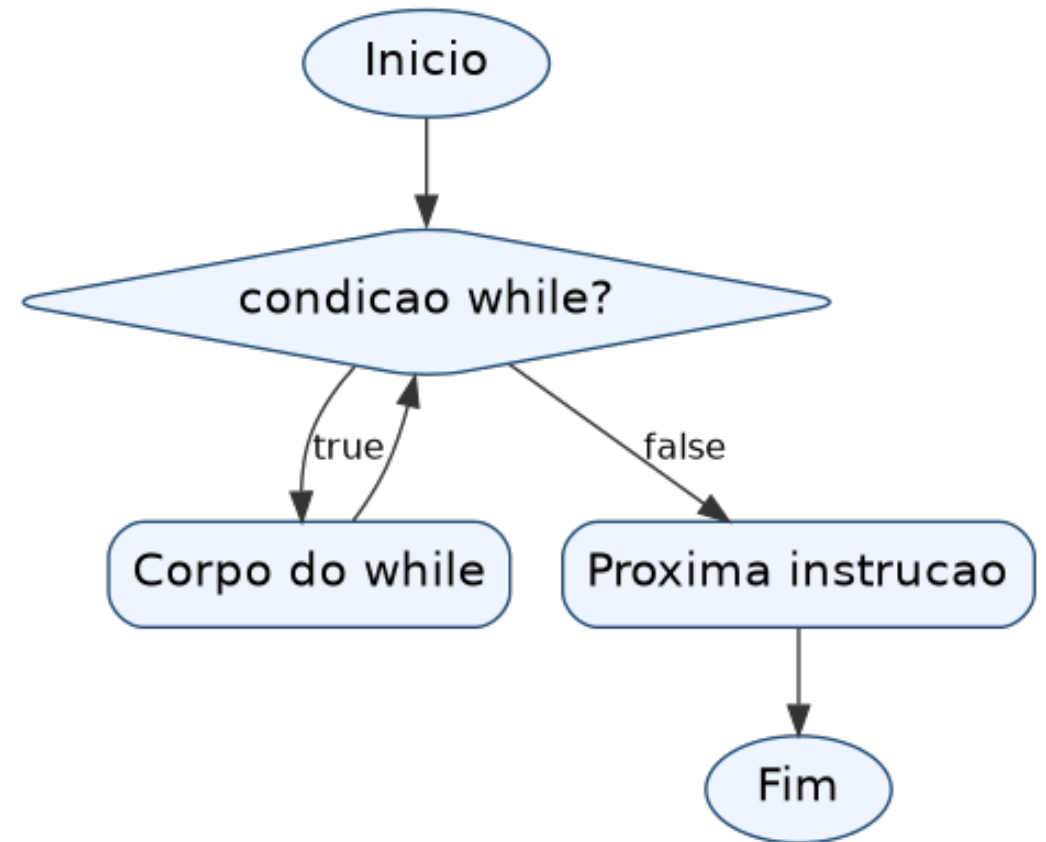
Apos os blocos, o fluxo normalmente se encontra novamente.

Representacao 4 - while

No while, a condicao e avaliada antes do corpo.

Por isso, o corpo pode executar zero vezes.

```
while (i < n) {  
    soma += i;  
    i++;  
}  
return soma;
```



Leitura estrutural do while

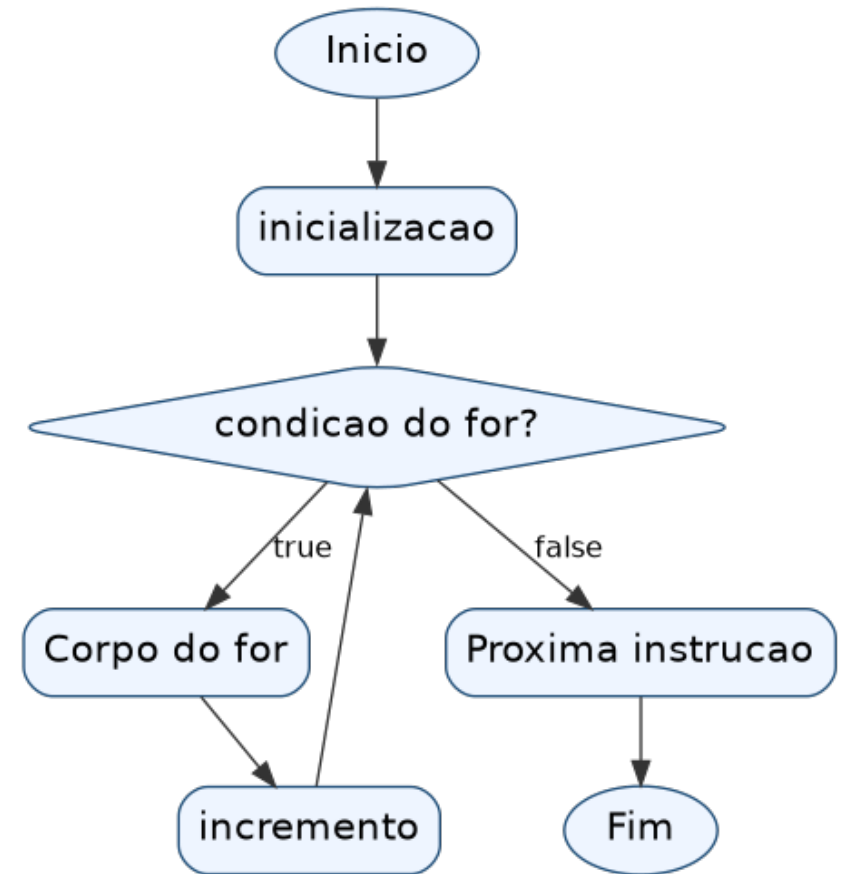
- Se a condicao for falsa no inicio, o corpo nao executa.
- Se for verdadeira, entra no corpo.
- Ao final do corpo, volta para a condicao.

Casos importantes: 0 iteracoes, 1 iteracao, mais de 1 iteracao.

Representacao 5 - for

O for pode ser visto como uma combinacao de inicializacao, condicao, corpo e incremento.

```
for (int i = 0; i < n; i++) {  
    total += i;  
}  
return total;
```



Leitura estrutural do for

Estruturalmente, o for possui uma decisao principal: a condicao de permanencia no laco.

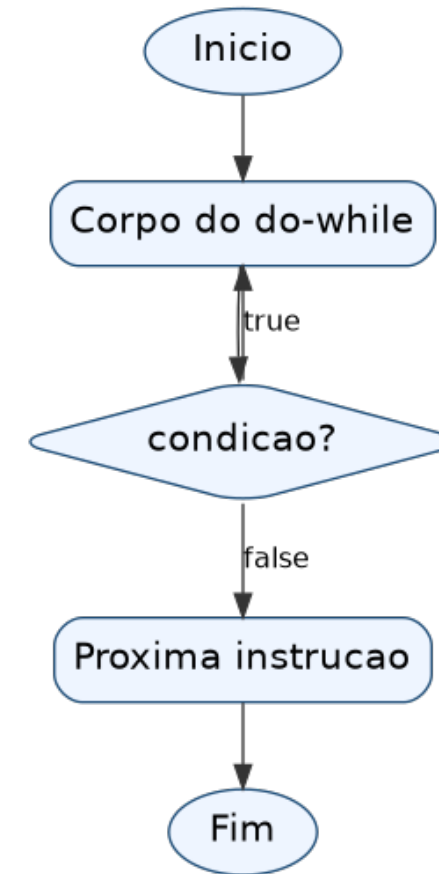
O incremento cria a aresta de retorno para a condicao.

Representacao 6 - do/while

No do/while, o corpo executa antes da primeira avaliacao da condicao.

Logo, o corpo executa pelo menos uma vez.

```
do {  
    total += i;  
    i++;  
} while (i < n);  
return total;
```



Leitura estrutural do do/while

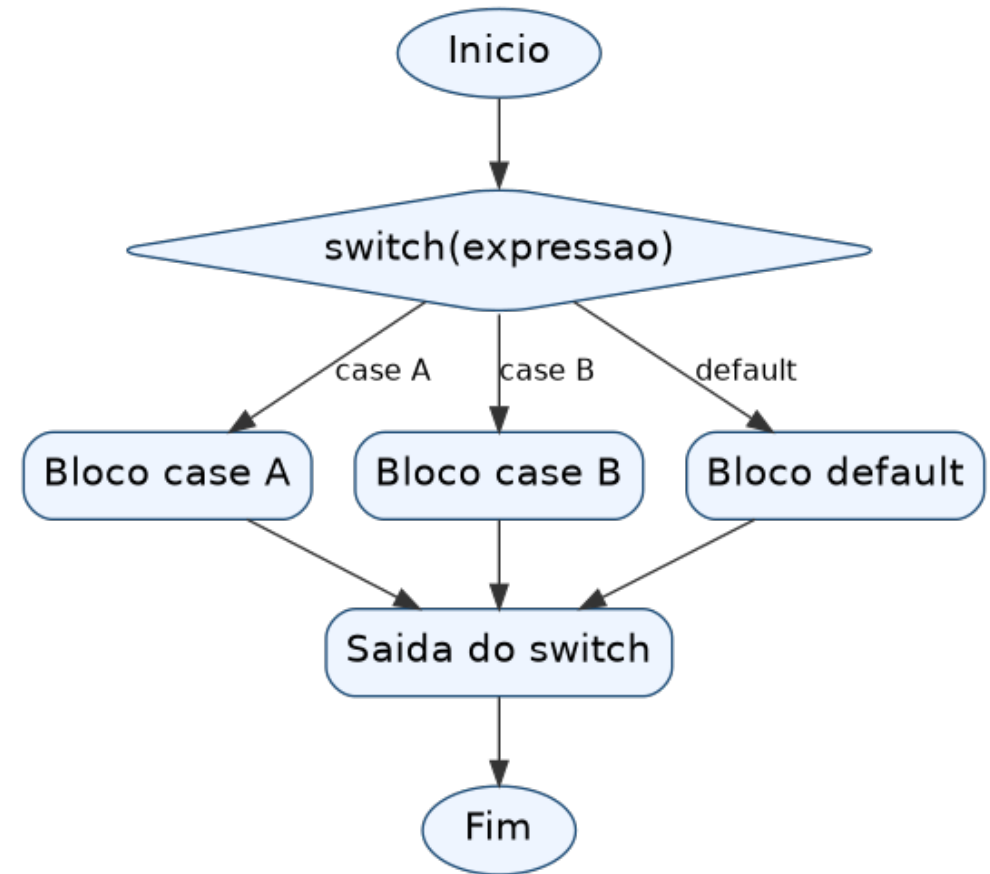
- Não existe caso com zero execuções do corpo.
- Existe caminho com uma execução.
- Existe caminho com mais de uma execução.

Isso diferencia o do/while e o do/for.

Representacao 7 - switch/case

O switch cria multiplas alternativas a partir de uma expressao.

```
switch (tipo) {  
    case 1: r = "A"; break;  
    case 2: r = "B"; break;  
    default: r = "Outro";  
}  
return r;
```



Leitura estrutural do switch

Cada case representa uma saída possível da decisão.

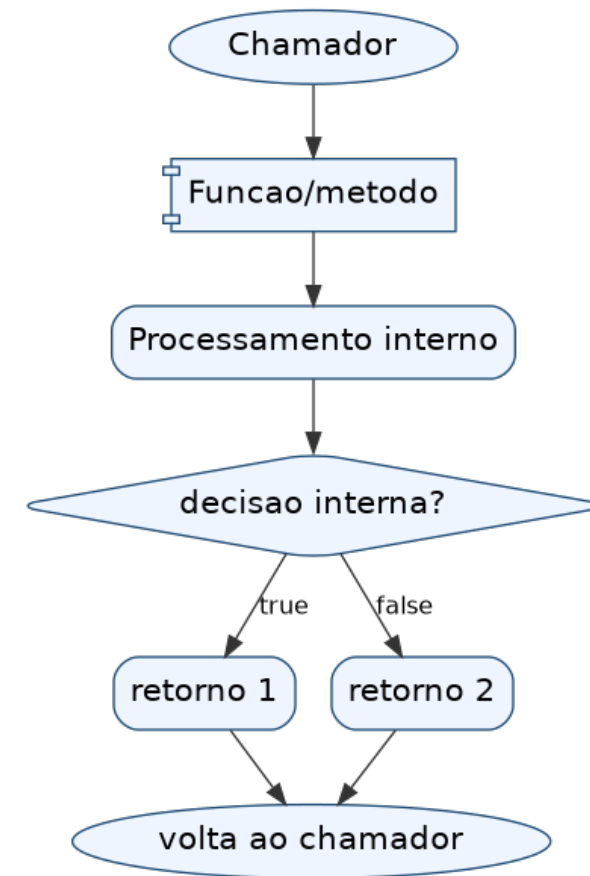
Para cobertura estrutural, normalmente queremos um caso de teste por alternativa relevante.

Cuidado: sem break, pode ocorrer queda de fluxo entre cases.

Representacao 8 - funcao/metodo

Uma funcao ou metodo pode ser visto como um subgrafo chamado por outro ponto do programa.

```
double calcularTotal(double valor) {  
    if (valor > 100) return valor * 0.9;  
    return valor;  
}
```



Leitura estrutural de funcao/metodo

- O metodo tem entrada.
- Pode possuir decisoes internas.
- Pode possuir multiplos retornos.
- Cada retorno representa uma saida possivel do subgrafo.

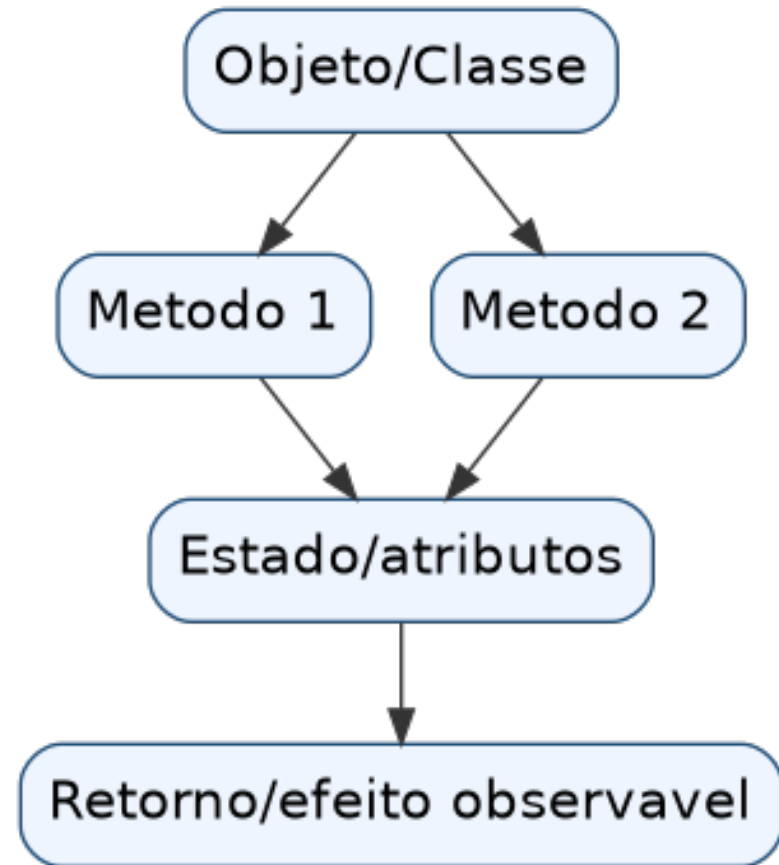
Em teste de unidade, geralmente testamos o metodo como unidade isolada.

Representacao 9 - classe

Uma classe nao e um unico caminho de execucao.

Ela organiza estado e comportamento.

```
class Carrinho {  
    private double total;  
  
    void adicionar(double valor) { ... }  
    double fecharCompra() { ... }  
}
```



Leitura estrutural de classe

Para teste estrutural, normalmente analisamos:

- cada metodo individualmente;
- interacoes entre metodos quando o estado da classe influencia o comportamento;
- caminhos causados por atributos internos.

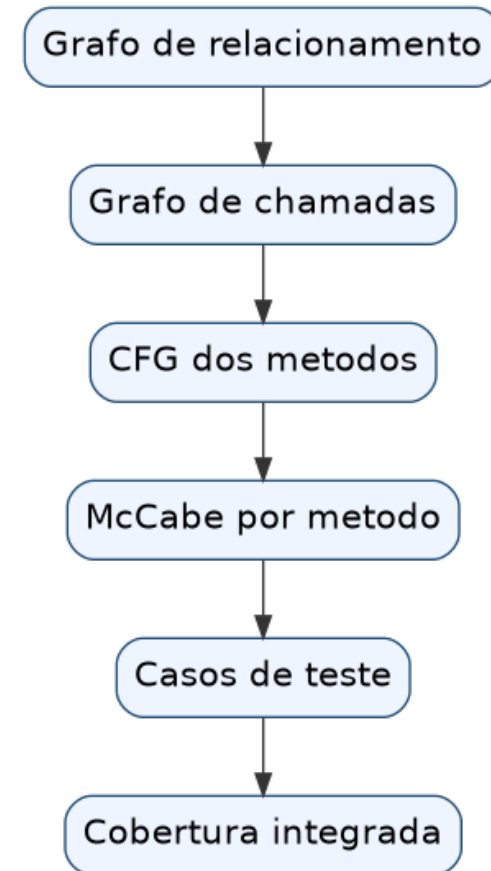
Classe -> metodos -> grafos de fluxo.

OO real - o que muda?

Em código orientado a objetos, normalmente não existe apenas um grafo.

Temos camadas de análise:

- grafo de relacionamento entre classes;
- grafo de chamadas entre métodos;
- grafo de fluxo de controle dentro de cada método;
- possível análise integrada quando um método
- chama outros métodos.

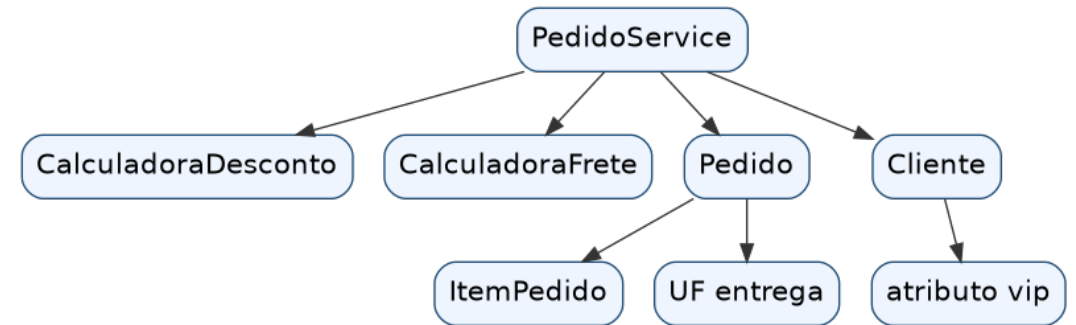


OO real - exemplo de dominio

Sistema simples de fechamento de pedido.

Classes envolvidas:

- PedidoService coordena o caso de uso;
- Pedido possui itens e UF de entrega;
- ItemPedido calcula seu total;
- Cliente informa se e VIP;
- CalculadoraDesconto aplica regras de desconto;
- CalculadoraFrete aplica regra por UF.



Codigo OO - classes principais

Observe que o comportamento esta distribuido entre objetos.

```
class Cliente {  
    boolean vip;  
}  
  
class ItemPedido {  
    double valor;  
    int quantidade;  
    double total() { return valor * quantidade; }  
}  
  
class Pedido {  
    List<ItemPedido> itens;  
    String uf;  
}
```

Codigo OO - regras distribuidas

As decisoes relevantes nao estao todas no mesmo metodo.

```
class CalculadoraDesconto {
    double calcular(Cliente c, double subtotal) {
        if (c.isVip()) return subtotal * 0.10;
        if (subtotal >= 500) return subtotal * 0.05;
        return 0;
    }
}

class CalculadoraFrete {
    double calcular(Pedido p) {
        switch (p.getUf()) {
            case "PR": return 20;
            case "SP": return 35;
            default: return 50;
        }
    }
}
```

Codigo OO - metodo de servico

Metodo que coordena os objetos. Este e o ponto de entrada do caso de uso.

```
class PedidoService {
    private final CalculadoraDesconto descontos;
    private final CalculadoraFrete fretes;

    double fecharPedido(Pedido p, Cliente c) {
        if (p.getItems().isEmpty()) {
            throw new IllegalArgumentException("pedido vazio");
        }

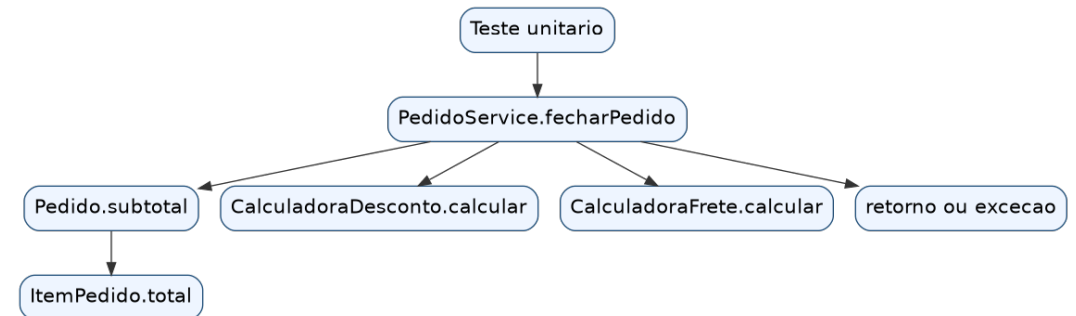
        double subtotal = p.subtotal();
        double desconto = descontos.calcular(c, subtotal);
        double frete = fretes.calcular(p);
        double total = subtotal - desconto + frete;

        if (total < 0) return 0;
        return total;
    }
}
```

OO - grafo de chamadas

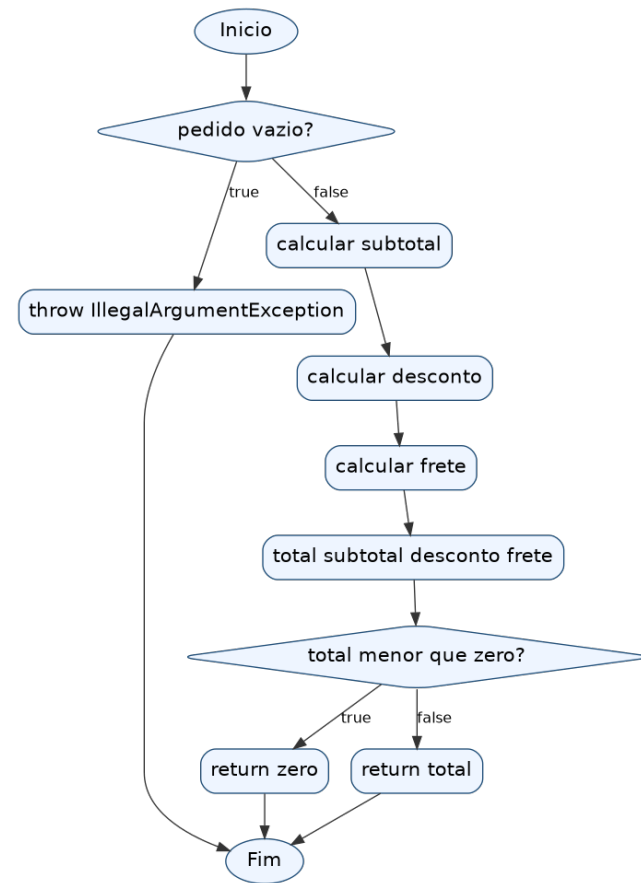
Antes do CFG detalhado, um código OO pode ser lido como grafo de chamadas.

Esse grafo mostra colaborações, mas ainda não mostra todos os caminhos internos.



OO - CFG do metodo de servico

Agora descemos para o fluxo de controle do metodo fecharPedido.



Solucao - McCabe do servico

Decisoes em fecharPedido:

D1: pedido vazio?

D2: total < 0?

$V(G) = \text{decisoes} + 1$

$V(G) = 2 + 1 = 3$

Caminhos independentes:

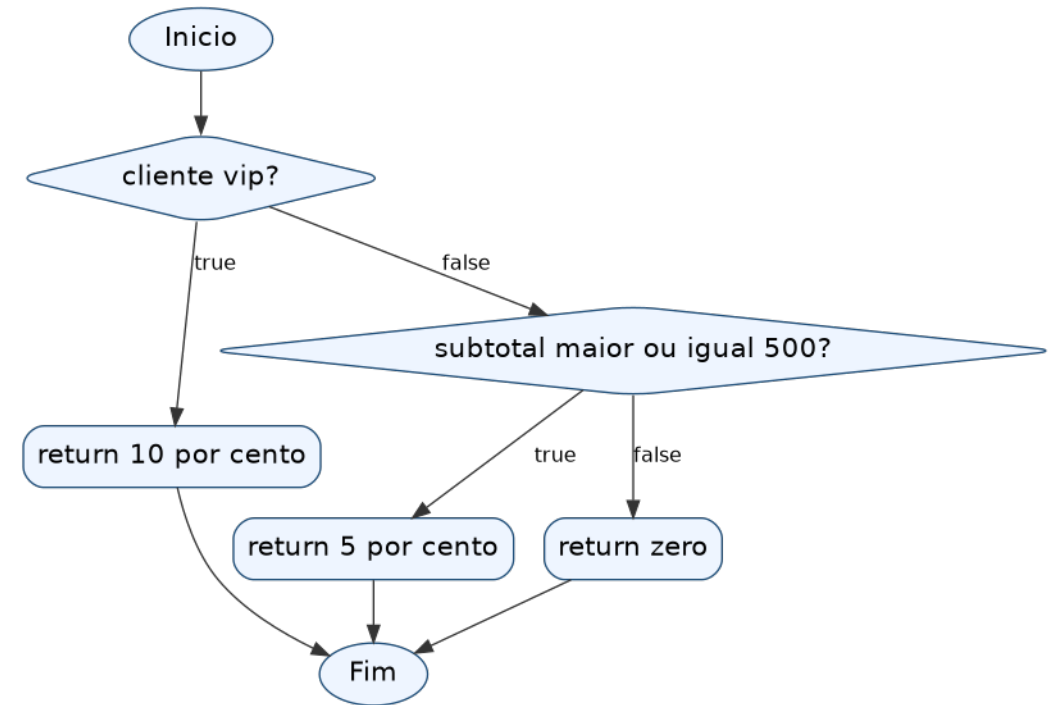
P1: pedido vazio -> execucao

P2: pedido nao vazio -> total < 0 -> return 0

P3: pedido nao vazio -> total >= 0 -> return total

OO - CFG da CalculadoraDesconto

A classe de desconto tem seu proprio grafo interno.



Solucao - McCabe do desconto

Decisoes em CalculadoraDesconto.calcular:

D1: cliente VIP?

D2: subtotal $\geq$ 500?

$V(G) = 3$

Caminhos:

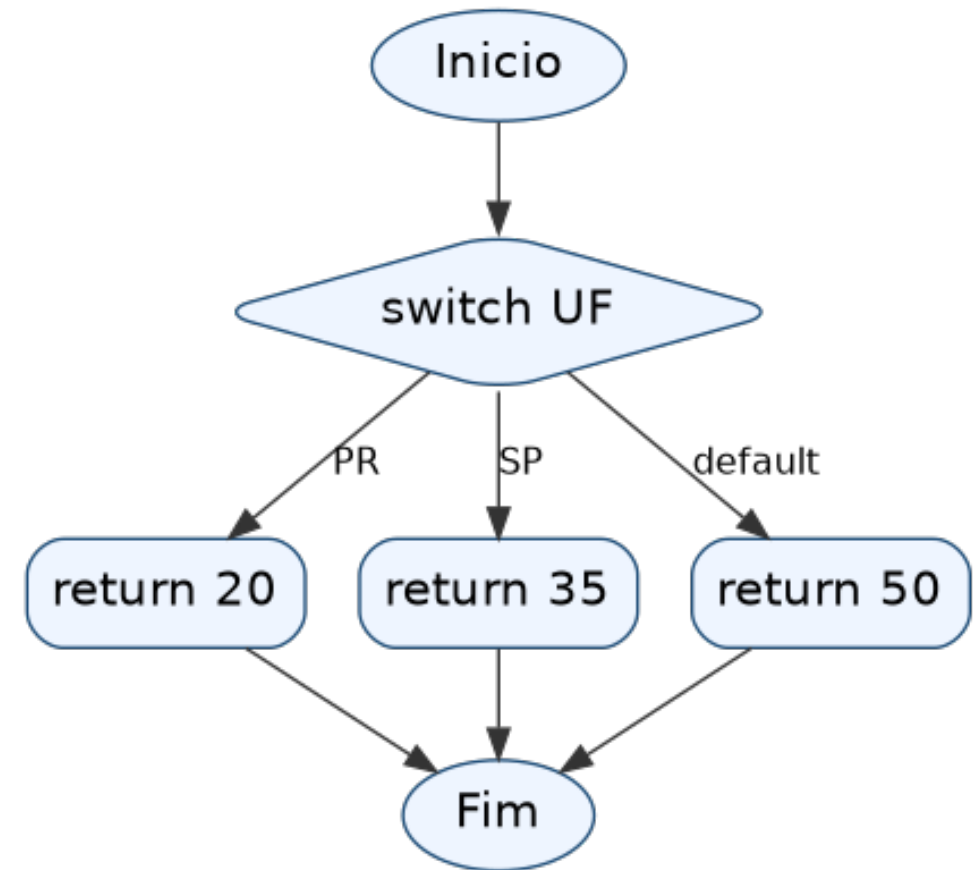
P1: VIP $\rightarrow$ 10%

P2: nao VIP e subtotal $\geq$ 500 $\rightarrow$ 5%

P3: nao VIP e subtotal $<$ 500 $\rightarrow$ 0

OO - CFG da CalculadoraFrete

O switch da UF também gera alternativas de fluxo.



Solucao - McCabe do frete

Para um switch com 3 saidas relevantes:

UF = PR -> frete 20

UF = SP -> frete 35

UF = outra -> frete 50

$V(G) = 3$

Na pratica: pelo menos um teste por alternativa relevante do switch.

OO - conclusao importante

No OO, o grafo de classes nao substitui o CFG.

Ele ajuda a localizar onde os CFGs estao.

Fluxo recomendado:

1. olhar as classes e dependencias;
2. identificar o metodo de entrada;
3. montar o grafo de chamadas;
4. abrir o CFG dos metodos com decisoes;
5. criar testes que cubram caminhos locais e colaboracoes.

OO - casos de teste sugeridos

CT01: pedido sem itens -> excecao

CT02: cliente comum, subtotal < 500, UF PR -> sem desconto, frete PR

CT03: cliente VIP, UF SP -> desconto VIP, frete SP

CT04: cliente comum, subtotal >= 500, UF RJ -> desconto por valor, frete default

CT05: forcar total < 0 com stub/mock de desconto -> return 0

OO - ponto didatico sobre testabilidade

O caminho total < 0 pode ser difícil de alcançar com as regras reais.

Isso mostra algo importante: algumas arestas do CFG podem exigir:

- dados especiais;
- objetos em estados específicos;
- stubs ou mocks;
- revisão da regra de negócio;
- remoção de código defensivo inalcançável.

Teste estrutural também revela problemas de projeto e testabilidade.

OO - exercicio em sala

Dado o codigo OO apresentado:

1. desenhe o grafo de relacionamento entre classes;
2. desenhe o grafo de chamadas a partir de fecharPedido;
3. desenhe o CFG de CalculadoraDesconto;
4. desenhe o CFG de CalculadoraFrete;
5. calcule $V(G)$ de cada metodo;
6. proponha testes unitarios e integrados;
7. indique quais caminhos exigem mock/stub.

Gabarito OO - resumo

PedidoService.fecharPedido:

$V(G) = 3$

CalculadoraDesconto.calcular:

$V(G) = 3$

CalculadoraFrete.calcular:

$V(G) = 3$

Pedido.subtotal:

possui loop sobre itens; $V(G) = 2$

Conclusao: a cobertura estrutural deve ser pensada por metodo e depois integrada pelo grafo de chamadas.

Regra mental

Ao ler um código, procure primeiro por estruturas que mudam o fluxo:

if, else, switch, for, while, do/while, return, break, continue, throw

Blocos basicos

Nem sempre precisamos transformar cada linha em um no.

Podemos agrupar instrucoes sequenciais em blocos basicos.

Um bloco basico e uma sequencia sem desvio interno.

Exemplo simples - condicao

Analise o codigo:

```
String classificar(int numero) {  
    if (numero >= 0) {  
        return "positivo";  
    }  
    return "negativo";  
}
```

Solucao do exemplo simples

Decisao: numero ≥ 0

Caminhos:

P1: condicao verdadeira $\rightarrow$ positivo

P2: condicao falsa $\rightarrow$ negativo

Casos:

CT1: numero = 10

CT2: numero = -10

McCabe: $1 + 1 = 2$

Estrutural x funcional

No teste funcional, talvez valores-limite sejam importantes: -1, 0, 1.

No teste estrutural inicial, a preocupacao e: a condicao assumiu true e false?

As duas visoes se complementam.

Exemplo - duas decisoes aninhadas

Analise:

```
String analisar(int x) {  
    if (x > 0) {  
        if (x % 2 == 0) {  
            return "positivo par";  
        }  
        return "positivo impar";  
    }  
    return "nao positivo";  
}
```

Solucao - duas decisoes aninhadas

Decisoes:

D1: $x > 0$

D2: $x \% 2 == 0$

McCabe: $2 + 1 = 3$

Caminhos:

P1: D1 true, D2 true

P2: D1 true, D2 false

P3: D1 false

Casos: $x=2$, $x=3$, $x=-1$

Complexidade Ciclomática de McCabe

McCabe fornece uma medida da complexidade lógica do fluxo de controle.

Ela indica o tamanho de uma base de caminhos linearmente independentes.

Formula principal

Para um grafo conectado:

$$V(G) = E - N + 2$$

Onde:

E = numero de arestas

N = numero de nos

Formula pratica

Em codigos estruturados simples, podemos conferir com:

$$V(G) = \text{numero de decisoes} + 1$$

Essa forma ajuda muito na aula, mas a formula do grafo continua sendo importante.

Caminho basico nao e caminho minimo

Use a expressao teste de caminho basico ou base minima de caminhos independentes.

Nao confundir com caminho minimo de algoritmos de grafos.

Aqui, minimo significa conjunto base suficiente para introduzir as arestas/caminhos independentes relevantes.

Caminhos independentes

Um caminho independente introduz pelo menos uma aresta ainda nao usada pelos caminhos anteriores.

A ideia nao e testar todas as combinacoes possiveis, mas montar uma base estrutural.

Exemplo guiado completo

Considere:

```
int desconto(int idade, boolean estudante) {  
    int desconto = 0;  
  
    if (idade >= 60) {  
        desconto = 20;  
    } else if (estudante) {  
        desconto = 10;  
    }  
  
    return desconto;  
}
```

Exemplo guiado - decisoes e McCabe

Decisoes:

D1: idade ≥ 60

D2: estudante

$$V(G) = 2 + 1 = 3$$

Precisamos de 3 caminhos independentes.

Exemplo guiado - caminhos e casos

P1: D1 true -> desconto 20

P2: D1 false, D2 true -> desconto 10

P3: D1 false, D2 false -> desconto 0

Casos:

CT1: idade=65, estudante=false, resultado=20

CT2: idade=20, estudante=true, resultado=10

CT3: idade=30, estudante=false, resultado=0

Exemplo - decisoes independentes

Analise:

```
int calcular(int x, int y) {  
    int resultado = 0;  
  
    if (x > 0) {  
        resultado += 10;  
    }  
  
    if (y > 0) {  
        resultado += 20;  
    }  
  
    return resultado;  
}
```

Solucao - decisoes independentes

Decisoes:

D1: $x > 0$

D2: $y > 0$

McCabe: 3

Combinacoes possiveis: 4

VV, VF, FV, FF

A base de caminhos pode ter 3 caminhos. O quarto pode ser util funcionalmente, mas nao e obrigatorio para a base.

Exemplo - loop

Analise:

```
int somar(int n) {  
    int soma = 0;  
  
    for (int i = 0; i < n; i++) {  
        soma += i;  
    }  
  
    return soma;  
}
```

Solucao - loop

Decisao: $i < n$

McCabe: $1 + 1 = 2$

Caminhos fundamentais:

P1: nao entrar no laco

P2: entrar no laco

Casos: $n=0$ e $n=1$

Discussao pratica: $n=2$ tambem pode ser util para testar multiplas iteracoes.

Exercicio 1 - aquecimento

Faca:

1. identifique decisoes;
2. desenhe o grafo;
3. calcule $V(G)$;
4. identifique caminhos;
5. crie casos de teste.

```
String temperatura(double temp) {  
    if (temp > 30) {  
        return "quente";  
    }  
    return "normal";  
}
```

Exercicio 2

Determine CFG, N, E, V(G), caminhos independentes e entradas.

```
String classificar(int valor) {  
    if (valor < 0) {  
        return "negativo";  
    } else if (valor == 0) {  
        return "zero";  
    } else {  
        return "positivo";  
    }  
}
```

Exercicio 3

Pergunta adicional: existem combinacoes possiveis que nao fazem parte da base?

```
double calcularFrete(double valor, boolean vip) {  
    double frete = 20;  
  
    if (valor >= 200) {  
        frete = 0;  
    }  
  
    if (vip) {  
        frete = frete * 0.5;  
    }  
  
    return frete;  
}
```

Exercicio 4

Determine N, E, V(G) e caminhos basicos.

```
int calcular(int x) {  
    int resultado = 0;  
  
    if (x > 10) {  
        if (x > 20) {  
            resultado = 2;  
        } else {  
            resultado = 1;  
        }  
    } else {  
        resultado = 0;  
    }  
  
    return resultado;  
}
```

Exercicio 5 - com repeticao

Identifique decisoes, construa o grafo, calcule McCabe e escolha valores de n.

```
int contador(int n) {  
    int total = 0;  
  
    for (int i = 0; i < n; i++) {  
        if (i % 2 == 0) {  
            total++;  
        }  
    }  
  
    return total;  
}
```

Exercicio 6 - desafio

Quantas decisoes? Qual $V(G)$? Quais caminhos independentes? Quais entradas?

```
double calcularValor(double valor, int quantidade, boolean clienteVip) {  
    double total = valor * quantidade;  
  
    if (quantidade >= 10) {  
        total = total * 0.9;  
    }  
  
    if (clienteVip) {  
        total = total * 0.95;  
    }  
  
    if (total > 1000) {  
        total = total - 50;  
    }  
  
    return total;  
}
```

Desafio final

Responda em 60 segundos:

- numero de decisoes
- $V(G)$
- numero de caminhos independentes

```
String avaliar(int nota, int faltas) {  
    if (faltas > 20) {  
        return "reprovado";  
    }  
  
    if (nota >= 70) {  
        return "aprovado";  
    } else if (nota >= 40) {  
        return "exame";  
    } else {  
        return "reprovado";  
    }  
}
```

Gabarito - Exercício 1

Decisoes: $\text{temp} > 30$

$V(G)=2$

Caminhos:

P1: $\text{temp} > 30$

P2: $\text{temp} \leq 30$

Casos: $\text{temp}=35$ e $\text{temp}=25$

Gabarito - Exercício 2

Decisões:

D1: valor < 0

D2: valor == 0

$V(G)=3$

Caminhos:

P1: valor < 0

P2: valor ≥ 0 e valor == 0

P3: valor ≥ 0 e valor != 0

Entradas: -1, 0, 1

Gabarito - Exercício 3

D1: valor ≥ 200

D2: vip

$V(G)=3$

Base possível:

P1: D1 false, D2 false

P2: D1 true, D2 false

P3: D1 false, D2 true

Caminho adicional possível: D1 true, D2 true

Gabarito - Exercício 4

D1: $x > 10$

D2: $x > 20$

$V(G)=3$

Caminhos:

P1: $x \leq 10$

P2: $x > 10$ e $x \leq 20$

P3: $x > 20$

Casos: $x=5$, $x=15$, $x=25$

Gabarito - Exercício 5

D1: $i < n$

D2: $i \% 2 == 0$

$V(G)=3$

Base:

P1: não entrar no for

P2: entrar e if verdadeiro

P3: entrar e if falso

Valores interessantes: $n=0$, $n=1$, $n=2$

Gabarito - Exercício 6

D1: quantidade ≥ 10

D2: clienteVip

D3: total > 1000

$V(G)=4$

Não é necessário testar todas as 8 combinações para a base estrutural.
Mas combinações adicionais podem ser relevantes por razões funcionais.

Gabarito - Desafio final

D1: faltas > 20

D2: nota $\geq$ 70

D3: nota $\geq$ 40

$V(G)=4$

Caminhos:

P1: faltas > 20

P2: faltas $\leq$ 20 e nota $\geq$ 70

P3: faltas $\leq$ 20 e nota < 70 e nota $\geq$ 40

P4: faltas $\leq$ 20 e nota < 40

Fechamento

Habilidade central:

Codigo -> Grafo -> Complexidade -> Caminhos -> Entradas -> Testes

Quando o aluno imagina o grafo antes de desenha-lo, ele comecou a analisar estruturalmente o software.